Queue ADT, Operations & Array Implementation

Q. Define Queue as an Abstract Data Type (ADT). Explain its standard operations and demonstrate its implementation using an Array with a relevant code snippet.

> **Definition to Remember**

> A **(

be rem

---

1. Queue as an ADT

As an Abstract Data Type, a Queue manages a collection of elements and restricts access to two specific ends:

- **R

2. Standard Operations

| Operation | Description | Condition |

| :--- | :

3. Array Implementation in C

Two pointers (`front` and `rear`) are initialized to `-1`.

```c

#incluc

int queue[SIZE];

int fron

void enqueue(int value) {

if (rear

int dequeue() {

if (fron
```

4. Advantages and Limitations

- **Advantage:** Array implementation is simple and fast (O(1) time complexity for enqueue/dequeue).

- **A

> **Must-Write Points (for 10 marks)**

> 1. Q

> **Quick Recall**

> `Que
array v

Complete Executable C Code: Linear Queue Array Implementation

```c

#incluc

#define MAX 5

struct Queue {

int iten

void initQueue(struct Queue *q) {

q->fron

int isFull(struct Queue *q) {

return
```

```
int isEmpty(struct Queue *q) {
return

void enqueue(struct Queue *q, int val) {
if (isFu

int dequeue(struct Queue *q) {
if (isEn

void display(struct Queue *q) {
if (isEn

int main() {
struct

enqueue(&q, 10);
enque

printf("Dequeued: %d\n", dequeue(&q));
display

return 0;
}
```